



how to start my ai training for a music ai model

The best way to start is not to train a music model from scratch. Start by fine-tuning an existing model—most practically **MusicGen**—on a carefully prepared, legally usable dataset, then expand toward song structure, tempo control, and storytelling. MusicGen is a single-stage Transformer that generates compressed audio tokens through EnCodec and supports text and melody conditioning. ^[1]

Recommended first target

For your goals—intro, verse, chorus, outro, beat awareness, and full song ideas—use a staged architecture:

Stage	Model approach	Purpose
1	MusicGen fine-tuning	Learn your musical style and prompt vocabulary
2	Music metadata pipeline	Add BPM, key, genre, instruments, mood, and structure labels
3	Section generation	Generate separate intro, verse, chorus, and outro segments
4	Arrangement engine	Join sections while preserving tempo and key
5	Long-form system	Generate complete songs using planning plus audio generation

MusicGen itself is limited to approximately 30 seconds per generation in the Hugging Face implementation, so full-song generation should initially be handled as multiple coordinated sections rather than one long generation.

Use your workstation correctly

Use your MacBook Pro as the **control plane**:

- Dataset organization and cleaning.
- Metadata generation.
- Audio inspection.
- Prompt design.
- Experiment configuration.
- Git and SSH orchestration.
- Lightweight inference and validation.

Use the Ubuntu NVIDIA workstation as the **training plane**:

- Audio tokenization.
- Model fine-tuning.
- Multi-GPU experiments.
- Checkpoint storage.
- Evaluation generation.
- Long-running jobs.

Your 4 × RTX A6000 system gives you substantial VRAM, but begin with one GPU. Multi-GPU training adds distributed-systems complexity and should only be enabled after a single-GPU run is reproducible. Your local workstation configuration and Mac/Ubuntu split are already well suited to this workflow.

Choose the first model

Start with one of these:

- **MusicGen-small:** Best for validating your pipeline quickly.
- **MusicGen-medium:** Better quality and a reasonable next step on your A6000 workstation.
- **MusicGen-melody:** Appropriate if melody or reference-audio conditioning matters.
- **MusicGen-large:** Delay this until the dataset, preprocessing, evaluation, and checkpointing are proven.

AudioCraft provides MusicGen models at approximately 300M, 1.5B, and 3.3B parameter scales, and its documentation identifies medium or melody as a useful quality/compute trade-off.

For your first serious experiment, I would use:

```
facebook/musicgen-small
```

Then move to:

```
facebook/musicgen-medium
```

Do not begin with a randomly initialized model. Pretraining a high-quality music generator requires a very large licensed corpus, substantial compute, and a reliable audio-token training pipeline. AudioCraft reports that MusicGen was trained using 20,000 hours of licensed music and does not release that original training dataset.

Prepare legally usable audio

Only use audio for which you have appropriate rights or a license that permits machine-learning training. Maintain a dataset manifest recording:

- Source and license.
- Artist or rights holder.

- Track identifier.
- Date acquired.
- Permitted uses.
- Whether commercial use is allowed.
- Whether derivative model training is allowed.
- Removal or takedown status.

Avoid collecting commercial songs from streaming services or using copyrighted music without permission. For a commercial fintech-grade or enterprise-grade workflow, treat dataset provenance as an auditable control, similar to a data lineage requirement.

A useful first dataset might contain:

- 50–200 tracks for a style experiment.
- Tracks longer than 30 seconds.
- Consistent or well-documented sample rates.
- Instrumental material initially, if you want to reduce lyric and vocal-rights complexity.
- Multiple arrangements within the target style.
- A separate validation set that is never used for training.

Small datasets can be useful for style adaptation, but they are not sufficient for teaching a general-purpose music model from scratch.

Create the audio layout

On the Ubuntu workstation, create a project structure like this:

```
mkdir -p ~/ai/projects/musicgen-style/{data/{raw,clean,train,val,test},metadata,conf:
cd ~/ai/projects/musicgen-style

find . -maxdepth 3 -type d | sort
```

Use these conventions:

data/raw/	Original source files; never modify
data/clean/	Normalized and validated audio
data/train/	Training audio
data/val/	Validation audio
data/test/	Final held-out evaluation audio
metadata/	JSON metadata and manifests
checkpoints/	Model checkpoints
outputs/	Generated WAV files
logs/	Training and evaluation logs
scripts/	Preprocessing and evaluation code
configs/	Reproducible experiment settings

Keep raw files immutable. Generate derived files into new directories so that preprocessing remains reproducible.

Normalize the audio

For the first MusicGen experiment, standardize:

```
Format: WAV or FLAC
Sample rate: 32 kHz
Channels: mono for the simplest first run
Segment duration: 30 seconds
Peak handling: avoid clipping
Loudness: normalize consistently
```

MusicGen's documented training configuration uses a 32 kHz EnCodec music tokenizer with four codebooks sampled at 50 Hz.

Example FFmpeg normalization:

```
ffmpeg -i input.wav \
  -map_metadata -1 \
  -ac 1 \
  -ar 32000 \
  -c:a pcm_s16le \
  output.wav
```

For production-quality preprocessing, also calculate:

- Duration.
- RMS or LUFS loudness.
- Peak amplitude.
- Silence ratio.
- Clipping ratio.
- Sample rate.
- Channel count.
- BPM.
- Musical key.
- Instrument labels.
- Section boundaries.

Do not blindly normalize every track to the same loudness if dynamics are an important part of the musical style. Instead, preserve the original file and record the transformation in metadata.

Build metadata

AudioCraft expects music metadata to be stored in JSON files associated with the audio data.

A useful metadata record could look like this:

```
{
  "path": "data/clean/track_001.wav",
  "duration": 30.0,
  "sample_rate": 32000,
  "channels": 1,
  "bpm": 96,
  "key": "A minor",
  "genre": "cinematic hip hop",
  "mood": ["dark", "determined", "emotional"],
  "instruments": ["piano", "808 bass", "strings", "drums"],
  "section": "chorus",
  "energy": 0.82,
  "description": "Dark cinematic hip hop chorus with piano, heavy 808 bass, wide str",
  "license": "owned",
  "source_id": "track_001"
}
```

For your intended section-based system, add structural fields:

```
{
  "song_id": "song_001",
  "section_index": 2,
  "section_type": "chorus",
  "start_time": 48.0,
  "end_time": 78.0,
  "bpm": 96,
  "key": "A minor",
  "energy": 0.9,
  "narrative_role": "central emotional hook"
}
```

The description field is especially important because the model learns the relationship between text conditioning and audio. Use consistent vocabulary rather than randomly varying descriptions.

Start with a baseline

Before fine-tuning, verify that the pretrained model works. On the Ubuntu workstation:

```
conda create -n musicgen python=3.10 -y
conda activate musicgen

python -m pip install --upgrade pip
python -m pip install torch torchaudio
python -m pip install transformers datasets soundfile scipy
```

Then test model loading:

```
python - <<'PY'
import torch
from transformers import AutoProcessor, MusicgenForConditionalGeneration

device = "cuda" if torch.cuda.is_available() else "cpu"
model_name = "facebook/musicgen-small"

processor = AutoProcessor.from_pretrained(model_name)
model = MusicgenForConditionalGeneration.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32
).to(device)

inputs = processor(
    text=["cinematic hip hop track with piano, deep bass, and dramatic drums"],
    padding=True,
    return_tensors="pt"
).to(device)

with torch.inference_mode():
    audio = model.generate(
        **inputs,
        do_sample=True,
        guidance_scale=3,
        max_new_tokens=256
    )

print("device:", device)
print("audio shape:", tuple(audio.shape))
print("sample rate:", model.config.audio_encoder.sampling_rate)
PY
```

MusicGen supports text-conditioned and audio-prompted generation, and sampling generally produces better results than greedy decoding.

Fine-tune rather than pretrain

AudioCraft provides a MusicGen training pipeline based on autoregressive prediction over discrete audio-token streams. It supports continuing from pretrained MusicGen checkpoints using the `continue_from` configuration.

The conceptual command is:

```
dora run solver=musicgen/musicgen_base_32khz \
    model/lm/model_scale=small \
    continue_from=//pretrained/facebook/musicgen-small \
    conditioner=text2music
```

Before running it, follow the exact AudioCraft environment setup for the version you install because configuration names and dependencies can change. The official repository warns that

configuration is not automatically inherited when continuing from a checkpoint, so the model scale, conditioner, tokenizer, and channel configuration must remain compatible.

For your first training experiment:

```
Model: MusicGen-small
Tokenizer: pretrained EnCodec 32 kHz
Channels: mono
Conditioner: text2music
Segment length: 30 seconds
Training: one GPU
Checkpointing: frequent
Validation: fixed prompts and fixed audio set
```

After the pipeline works, test `medium` and then melody conditioning.

Do not start with song sections alone

If you train only on files labeled “chorus,” the model may learn a chorus-like timbre but not necessarily understand how a chorus functions in a song. Build a balanced dataset containing:

- Intro.
- Verse.
- Pre-chorus.
- Chorus.
- Bridge.
- Outro.
- Instrumental transition.
- Full-song context where licensing permits.

Then use prompts such as:

```
intro, 92 BPM, A minor, sparse piano, atmospheric pads, low energy
verse, 92 BPM, A minor, restrained drums, narrative mood, medium energy
chorus, 92 BPM, A minor, wide strings, heavy drums, memorable melodic hook
outro, 92 BPM, A minor, reduced drums, fading piano, reflective mood
```

For long-form generation, your application should first create a **song plan**, then generate each section:

```
Song plan
-> tempo and key
-> emotional arc
-> section sequence
-> section prompts
-> generated audio sections
-> crossfades and beat alignment
```

```
-> loudness mastering
-> final validation
```

This planner can initially be a Python service or language model rather than part of the audio model itself.

Add tempo and beat control

BPM should be included in conditioning text, but text alone may not guarantee exact beat alignment. Add a post-generation validation layer that:

- Detects the generated BPM.
- Measures deviation from the target.
- Time-stretches only within an acceptable range.
- Rejects outputs with severe drift.
- Aligns section boundaries to musical bars.
- Preserves key where possible.
- Applies short equal-power crossfades.

For example, with a 4/4 time signature, a 96 BPM bar lasts:

$$\frac{4 \times 60}{96} = 2.5 \text{ seconds}$$

Generate or trim section boundaries to whole-bar durations whenever possible. This makes transitions more predictable.

Evaluate systematically

Do not evaluate only by listening to a few exciting samples. Create a fixed evaluation suite:

- Ten prompts per genre.
- Five BPM values.
- Several keys.
- Multiple section types.
- Repeated random seeds.
- At least one prompt outside the training distribution.

Track:

Metric	What it measures
Prompt adherence	Whether the result matches genre, mood, and instruments
Tempo accuracy	Difference between requested and detected BPM
Structural usefulness	Whether the section behaves like an intro, verse, chorus, or outro
Audio quality	Artifacts, clipping, noise, and distortion

Metric	What it measures
Diversity	Variation across seeds
Memorability	Human rating of musical appeal
Leakage	Similarity to training tracks or recognizable copyrighted material

Store every evaluation item:

```

experiment_id
checkpoint
prompt
seed
BPM target
detected BPM
audio path
human rating
automated metrics
notes

```

MLflow or Weights & Biases would be suitable for tracking experiments, while generated audio should be stored with immutable checkpoint references.

Suggested first milestone

Your first milestone should be:

Fine-tune MusicGen-small on a legally owned or licensed dataset and generate coherent 10–30 second sections in one target style with prompt-controlled BPM, mood, and instrumentation.

A practical sequence is:

1. Install and verify CUDA, PyTorch, and AudioCraft.
2. Generate baseline samples from the pretrained model.
3. Prepare 50–200 licensed or owned tracks.
4. Normalize and segment the audio.
5. Generate metadata and validate every record.
6. Create train, validation, and test splits by song, not by random segment.
7. Run a short single-GPU fine-tuning job.
8. Generate fixed validation prompts.
9. Compare the fine-tuned model against the base model.
10. Add section labels and BPM-aware post-processing.
11. Move to MusicGen-medium.
12. Add multi-GPU training only after reproducibility is established.

Do not let segments from the same song appear in both training and validation; that would make your evaluation overly optimistic. Also keep a permanently untouched test set for comparing checkpoints.

Your immediate next step

Start with **MusicGen-small, mono, 32 kHz, 30-second segments, one GPU, and a narrow musical style**. This gives you the fastest route to discovering whether your dataset, metadata, preprocessing, and training loop are working before you invest in large-scale multi-GPU training. AudioCraft supports cached EnCodec tokens, evaluation generation, and checkpoint continuation, which are useful once the initial pipeline is stable.

Sources

- [AudioCraft MusicGen documentation](#)
- [Hugging Face MusicGen documentation](#)
- [Hugging Face audio datasets documentation](#) ^[2]

*
**

what do i need to do my masc as my ubuntu workstation is getting frepaired atm

While your Ubuntu workstation is being repaired, use your MacBook Pro M3 Max as the **development, dataset, experimentation, and small-training machine**. It can run Apple's MLX and PyTorch through Metal/MPS, but it is not a replacement for the workstation's multi-GPU CUDA training capacity. ^{[16] [17]}

What the Mac should do

Use the Mac for:

- Preparing and cleaning music datasets.
- Converting and validating audio files.
- Generating BPM, key, loudness, and instrument metadata.
- Creating train/validation/test splits.
- Writing training and preprocessing scripts.
- Running small MusicGen or MLX experiments.
- Generating short samples for testing.
- Designing prompts and section labels.
- Tracking experiments with Git and MLflow.
- Preparing everything so training can resume on Ubuntu later.

Avoid using it initially for:

- Training MusicGen-medium or large models for long periods.
- Multi-GPU or distributed training.
- Large-scale audio-token preprocessing.
- Full-song generation pipelines.
- Large hyperparameter sweeps.
- Long-running jobs that consume all 48 GB of unified memory.

Your existing workflow already fits this division: the Mac acts as the control and preparation environment, while the Ubuntu machine performs serious CUDA workloads.

Install the Mac tools

First install Apple's command-line tools:

```
xcode-select --install
```

Install Homebrew if it is not already present:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Install core development tools:

```
brew update
brew install git git-lfs ffmpeg jq wget tree tmux uv
git lfs install
```

Install useful audio and system tools:

```
brew install sox libsndfile
```

Install VS Code:

```
brew install --cask visual-studio-code
```

The current Apple PyTorch guidance requires Apple Silicon, a recent macOS version, Python 3.10 or later, and Xcode command-line tools for MPS acceleration.^[17]

Create the project

Create a dedicated music AI project:

```
mkdir -p ~/ai/projects/musicgen-style/{data/{raw,clean,train,val,test},metadata,conf:
cd ~/ai/projects/musicgen-style
```

Your Mac project should look like this:

```
musicgen-style/  
├─ configs/  
├─ data/  
│   ├─ raw/  
│   │   ├─ clean/  
│   │   │   ├─ train/  
│   │   │   └─ val/  
│   └─ test/  
├─ metadata/  
├─ checkpoints/  
├─ logs/  
├─ notebooks/  
├─ outputs/  
└─ scripts/
```

Keep `data/raw` unchanged. Every preprocessing operation should create files under `data/clean`, which preserves reproducibility and makes it easier to identify mistakes.

Create Python environments

Use `uv` to create a fast, isolated Python environment:

```
cd ~/ai/projects/musicgen-style  
  
uv venv --python 3.11 .venv  
source .venv/bin/activate  
  
python -m pip install --upgrade pip
```

Install the general audio and dataset tools:

```
uv pip install \  
    numpy \  
    scipy \  
    pandas \  
    soundfile \  
    librosa \  
    audioread \  
    pydub \  
    tqdm \  
    pyyaml \  
    jsonlines \  
    datasets \  
    transformers \  
    accelerate \  
    safetensors \  
    jupyterlab \  
    matplotlib
```

Install PyTorch with Apple Silicon support:

```
uv pip install torch torchvision torchaudio
```

PyTorch uses the MPS backend to execute supported operations on Apple Silicon through Metal. [\[16\]](#) [\[17\]](#)

Install MLX for Apple Silicon experiments:

```
uv pip install mlx mlx-lm
```

MLX is Apple's machine-learning framework designed for Apple Silicon and is available through PyPI. [\[18\]](#) [\[19\]](#)

Save the environment:

```
uv pip freeze > requirements-mac.txt
```

Verify MPS

Run this test:

```
cd ~/ai/projects/musicgen-style
source .venv/bin/activate

python - <<'PY'
import torch

print("PyTorch:", torch.__version__)
print("MPS built:", torch.backends.mps.is_built())
print("MPS available:", torch.backends.mps.is_available())

if torch.backends.mps.is_available():
    x = torch.ones((4096, 4096), device="mps")
    y = x @ x
    print("MPS test passed:", y.shape)
else:
    print("MPS is unavailable")
PY
```

You want to see:

```
MPS built: True
MPS available: True
MPS test passed: torch.Size([4096, 4096])
```

If MPS is unavailable, check that you are using an arm64 Python:

```
python -c "import platform; print(platform.machine())"
```

The expected output is:

```
arm64
```

Also check your macOS version:

```
sw_vers
```

PyTorch documents `torch.backends.mps.is_available()` as the normal availability check for the MPS backend. ^[16]

Prepare your music data

Start with a small, legally usable dataset:

```
50-200 tracks
One target style
Owned or properly licensed audio
Consistent metadata
Separate validation tracks
No train/validation leakage
```

For each track, record:

- File path.
- License and source.
- Duration.
- Sample rate.
- Channels.
- BPM.
- Key.
- Genre.
- Mood.
- Instruments.
- Energy.
- Section type.
- Narrative role.
- Description used as the prompt.

For example:

```
{
  "path": "data/clean/track_001.wav",
```

```

    "duration": 30.0,
    "sample_rate": 32000,
    "channels": 1,
    "bpm": 96,
    "key": "A minor",
    "genre": "cinematic hip hop",
    "mood": ["dark", "emotional"],
    "instruments": ["piano", "strings", "808 bass", "drums"],
    "section": "chorus",
    "energy": 0.85,
    "description": "Dark cinematic hip hop chorus with piano, wide strings, deep 808 bass",
    "license": "owned",
    "source_id": "track_001"
}

```

For your long-term goal, add section labels such as:

```

intro
verse
pre-chorus
chorus
bridge
outro
instrumental-transition

```

Hugging Face provides dataset tooling for audio repositories and audio columns, which you can use later when moving from local files to a versioned dataset repository. [\[20\]](#)

Normalize audio on the Mac

Use FFmpeg to create a normalized copy:

```

mkdir -p data/clean

ffmpeg -i data/raw/input.wav \
    -map_metadata -1 \
    -ac 1 \
    -ar 32000 \
    -c:a pcm_s16le \
    data/clean/input.wav

```

For batch conversion:

```

find data/raw -type f \( -iname "*.wav" -o -iname "*.mp3" -o -iname "*.flac" -o -iname "*.aiff" \)
while IFS= read -r -d ' ' file; do
    name="$(basename "${file%.*}")"
    ffmpeg -y -i "$file" \
        -map_metadata -1 \
        -ac 1 \
        -ar 32000 \
        -c:a pcm_s16le \
        data/clean/${name}.wav
done

```

```
"data/clean/${name}.wav"  
done
```

Keep the original files in data/raw. Do not overwrite them.

Validate the audio

Create a validation script:

```
cat > scripts/validate_audio.py <<'PY'  
from pathlib import Path  
import json  
import soundfile as sf  
  
root = Path("data/clean")  
results = []  
  
for path in sorted(root.glob("*.wav")):  
    try:  
        info = sf.info(path)  
        record = {  
            "path": str(path),  
            "duration": round(info.duration, 3),  
            "sample_rate": info.samplerate,  
            "channels": info.channels,  
            "frames": info.frames,  
            "format": info.format,  
            "subtype": info.subtype,  
            "valid": True  
        }  
    except Exception as exc:  
        record = {  
            "path": str(path),  
            "valid": False,  
            "error": str(exc)  
        }  
  
    results.append(record)  
  
Path("metadata").mkdir(exist_ok=True)  
  
with open("metadata/audio_inventory.json", "w") as f:  
    json.dump(results, f, indent=2)  
  
valid = [x for x in results if x["valid"]]  
invalid = [x for x in results if not x["valid"]]  
  
print(f"Valid files: {len(valid)}")  
print(f"Invalid files: {len(invalid)}")  
PY  
  
python scripts/validate_audio.py
```

Inspect the results:


```
jq 'group_by(.sample_rate) | map({sample_rate: .[^2_0].sample_rate, count: length})'
```

Segment audio into examples

Use 30-second segments initially because MusicGen generation and training are naturally oriented around short audio windows. MusicGen uses the EnCodec audio tokenizer, and the documented setup uses 32 kHz audio processing.

Create a segmenting script:

```
cat > scripts/split_audio.sh <<'SH'
#!/usr/bin/env bash
set -euo pipefail

mkdir -p data/train

for file in data/clean/*.wav; do
    name="$(basename "${file%.wav}")"

    ffmpeg -y -i "$file" \
        -f segment \
        -segment_time 30 \
        -reset_timestamps 1 \
        -ac 1 \
        -ar 32000 \
        -c:a pcm_s16le \
        "data/train/${name}_%03d.wav"
done
SH

chmod +x scripts/split_audio.sh
./scripts/split_audio.sh
```

Do not randomly split segments from the same song into training and validation. Split by complete song first; otherwise, the validation score will be misleading.

Run small Mac experiments

Use the Mac to test:

- Audio loading.
- Tokenizer loading.
- Prompt formatting.
- Dataset validation.
- Short generation.
- Evaluation scripts.
- Checkpoint loading.
- Section concatenation.

- BPM detection.

Start with pretrained MusicGen inference rather than fine-tuning. The Hugging Face MusicGen implementation supports text descriptions and generation through the Transformers API.

Create a basic test:

```
cat > scripts/test_musicgen.py <<'PY'
import torch
import soundfile as sf
from transformers import AutoProcessor, MusicgenForConditionalGeneration

model_name = "facebook/musicgen-small"
device = "mps" if torch.backends.mps.is_available() else "cpu"

processor = AutoProcessor.from_pretrained(model_name)
model = MusicgenForConditionalGeneration.from_pretrained(model_name).to(device)

prompt = (
    "cinematic hip hop chorus, 96 BPM, A minor, "
    "dark piano, deep 808 bass, wide strings, powerful drums"
)

inputs = processor(
    text=[prompt],
    padding=True,
    return_tensors="pt"
).to(device)

with torch.inference_mode():
    audio_values = model.generate(
        **inputs,
        do_sample=True,
        guidance_scale=3,
        max_new_tokens=256
    )

audio = audio_values[0].cpu().numpy()
sample_rate = model.config.audio_encoder.sampling_rate

sf.write("outputs/musicgen_test.wav", audio.T, sample_rate)
print(f"Saved outputs/musicgen_test.wav using {device}")
PY

python scripts/test_musicgen.py
```

If it uses too much memory or runs slowly, reduce `max_new_tokens`, use a smaller model, or run preprocessing only. Do not repeatedly force the Mac into swap-heavy workloads.

Focus on dataset quality now

While the workstation is unavailable, the highest-value work is:

1. Define your target musical style.
2. Collect only permitted audio.
3. Normalize and validate every file.
4. Add BPM, key, mood, instrument, and section metadata.
5. Write consistent natural-language descriptions.
6. Create train, validation, and test splits.
7. Build automated checks.
8. Produce a small evaluation prompt set.
9. Commit scripts and configs to Git.
10. Package the project for transfer to Ubuntu.

The Mac can perform all of those tasks effectively and will save substantial time when the workstation returns.

Use Git for portability

Initialize the project:

```
cd ~/ai/projects/musicgen-style

git init
git lfs install

cat > .gitignore <<'EOF'
.venv/
__pycache__/
*.pyc
.DS_Store
data/raw/
data/clean/
data/train/
data/val/
data/test/
checkpoints/
outputs/*.wav
logs/
.env
EOF

git add scripts configs metadata requirements-mac.txt .gitignore
git commit -m "Initial music AI training project"
```

Do not commit copyrighted or private audio to Git. Use Git LFS only for files you are authorized to store, and keep large datasets in a separately controlled storage location.

Prepare for Ubuntu later

When the workstation is repaired, transfer the code and metadata:

```
rsync -avP \
  --exclude='.venv' \
  --exclude='data/raw' \
  --exclude='data/clean' \
  --exclude='checkpoints' \
  ~/ai/projects/musicgen-style/ \
  username@ubuntu-workstation:~/ai/projects/musicgen-style/
```

Then either regenerate normalized audio on Ubuntu or copy it separately:

```
rsync -avP \
  ~/ai/projects/musicgen-style/data/clean/ \
  username@ubuntu-workstation:~/ai/projects/musicgen-style/data/clean/
```

Copying metadata and scripts first is safer than immediately moving the entire dataset.
Confirm file counts and checksums before training:

```
find data/clean -type f -print0 | xargs -0 shasum -a 256 > metadata/clean_audio.sha256
```

On Ubuntu:

```
cd ~/ai/projects/musicgen-style
shasum -a 256 -c metadata/clean_audio.sha256
```

What to do today

Your immediate Mac plan should be:

- Install Homebrew, FFmpeg, Python, PyTorch, MLX, and dataset libraries.
- Create the project structure.
- Build the audio normalization and validation scripts.
- Prepare a small licensed dataset.
- Generate metadata for BPM, key, mood, instruments, and section.
- Run MusicGen-small inference on MPS.
- Create fixed evaluation prompts.
- Commit the code and metadata schemas.
- Wait for the repaired workstation before serious fine-tuning.

The Mac is fully capable of keeping the project moving, especially for preprocessing, validation, prompt design, and short inference. MPS can accelerate supported PyTorch

operations, while MLX is the more Apple-native option for experimentation on the M3 Max. ^[19]
^[18] ^[16]

✱✱

1. <https://github.com/facebookresearch/audiocraft/blob/main/docs/MUSICGEN.md>
2. <https://huggingface.co/docs/hub/en/datasets-audio>
3. <https://huggingface.co/blog/audio-datasets>
4. <https://huggingface.co/collections/amaai-lab/text-to-music>
5. <https://huggingface.co/sander-wood/text-to-music>
6. <https://huggingface.co/tencent/SongGeneration>
7. https://facebookresearch.github.io/audiocraft/api_docs/audiocraft/
8. <https://replicate.com/blog/fine-tune-musicgen>
9. <https://huggingface.co/spaces/facebook/MusicGen/blob/9cae843238aad3f5c7695a40c9ee77c42dd87aaf/docs/TRAINING.md>
10. <https://huggingface.co/datasets>
11. <https://huggingface.co/learn/audio-course/en/chapter4/fine-tuning>
12. <https://deepwiki.com/sakemin/cog-musicgen-chord/4.2-training-pipeline>
13. <https://huggingface.co/docs/datasets/index>
14. https://huggingface.co/docs/transformers/en/model_doc/musicgen
15. https://huggingface.co/docs/datasets/audio_process
16. <https://docs.pytorch.org/docs/stable/notes/mps.html>
17. <https://developer.apple.com/metal/pytorch/>
18. <https://github.com/ml-explore/mlx>
19. <https://developer.apple.com/videos/play/wwdc2025/315/>
20. <https://huggingface.co/docs/hub/en/datasets-audio>
21. <https://huggingface.co/docs/diffusers/en/optimization/mps>
22. <https://theashishmaurya.medium.com/i-ported-audiogen-to-apple-silicon-heres-everything-i-learned-7a6b7d4a2c84>
23. <https://github.com/pytorch/pytorch/wiki/MPS-Backend>
24. <https://discuss.pytorch.org/t/how-to-check-mps-availability/152015>
25. <https://blaiizzy.github.io/mlx-audio/>
26. <https://stackoverflow.com/questions/79124099/how-can-i-get-mps-running-in-pytorch-on-my-apple-m2>
27. <https://mlx-framework.org/>
28. <https://docs.pytorch.org/docs/stable/mps.html>
29. https://huggingface.co/docs/accelerate/en/usage_guides/mps
30. <https://opensource.apple.com/projects/mlx/>
31. <https://docs.pytorch.org/executorch/1.0/ios-mps.html>

